Project — Software Specification 26/27

A Verified Authorization Mini-Engine (VAME)

Instituto Superior Técnico | Bernardo Toninho | October 2026

*Groups of **2 students.** Submit via Fénix by the deadline shown on the course portal.*

Background and Motivation

Every cloud provider runs an authorization service: a piece of code that decides, given a request and a policy, whether the request should be allowed or denied. A wrong decision in either direction is, in the strict sense, a security incident. AWS publicly uses Dafny to verify portions of its IAM authorization stack; Cedar (the language and engine behind Amazon Verified Permissions) was designed from the start to be amenable to formal proof. Microsoft Research's IronFleet and IronClad projects use Dafny for security-critical distributed systems. Verified authorization is not a hypothetical use of the technique. It is shipping today.

In this project you will build, specify, and verify a miniature version of such an engine in two parts. **Part 1** is purely functional: datatypes, recursive evaluators, induction, lemmas. **Part 2** is imperative and object-oriented: a `PolicyStore` class with hidden representation, a decision cache backed by an array, and a batched-decision loop with a non-trivial invariant.

Once your Dafny code verifies, you will compile it to Python and run it under a small driver that loads a JSON test bundle and compares your decisions against an oracle.

> **Note on compilation targets.** Lab 7 walks through `dafny build -target cs` (C#) as a standalone exercise. This project uses `dafny build -target py` (Python) because Python requires no additional runtime and its `json` library simplifies the test-runner. Both are valid Dafny compilation targets, with the latter choice being practical. The supplied `Driver.py` and `tests/run_tests.py` scripts expect the Dafny Python build directory to be named `vame/`; the exact build and rename commands are given in the running-code component below.

Starter File and Shared Definitions

The file `vame.dfy` is provided as your starting point. It contains the following datatypes and stubs; **do not rename or remove anything that appears here** (the driver depends on these names). You may add helpers freely.

```
datatype Request = Req(user: string, action: string, resource: string
    )

datatype Matcher =
  | AnyReq
  | UserIs(name: string)
  | ActionIs(act: string)
  | ResourceIs(res: string)
  | And(left: Matcher, right: Matcher)
  | Or(left: Matcher, right: Matcher)
  | Not(inner: Matcher)

datatype Rule =
  | Allow(m: Matcher, priority: nat)
  | Deny(m: Matcher, priority: nat)
```

```
datatype Policy = Pol(rules: seq<Rule>)

datatype Decision = AllowD | DenyD | NoMatch
```

Semantics. A `Rule` says: *if the request matches `m`, then allow (or deny) it.* A `Policy` is a sequence of rules evaluated with *priority semantics*: the rule with the highest `priority` field among all rules whose matcher matches the request wins; ties are broken in favour of `Deny`. If no rule matches, the decision is `NoMatch`.

The starter also provides:

- `function matches(m: Matcher, r: Request): bool` — recursive matcher semantics. You may use it freely; you do not need to prove anything about it.
- `function rulePriority(rl: Rule): nat` — a convenience helper.
- `function ruleMatcher(rl: Rule): Matcher` — a convenience helper.
- `beats`, `combineBest`, `bestRule`, and `toDecision` — the fixed rule-selection interface, including supplied lemmas about membership, maximality, and appending a rule that cannot win. Proving this library is not a project task.
- `datatype Option<T> = None | Some(value: T)` — for the accumulator-style evaluator. Do **not** replace this with `Std.Wrappers.Option`: the compiled Python module's symbol names must remain stable for the driver to work.

Part 1 — Functional Core [10 val.]

1.1 Evaluator [1.5 val.]

Implement `eval(p: Policy, q: Request): Decision` as the public wrapper around the supplied `bestRule` selector: convert its `Option<Rule>` result with `toDecision`. The selection library fixes the priority and tie-breaking semantics; proving it is not a project task.

State and prove the *totality lemma* in the starter:

```
lemma eval_total(p: Policy, q: Request)
  ensures eval(p, q) == AllowD ||
          eval(p, q) == DenyD  ||
          eval(p, q) == NoMatch
```

This is a preliminary result mirroring the L1 worked example at slightly larger scale.

1.2 Stability under Low-Priority Additions [2 val.]

State and prove a *decision-stability* theorem:

> *Adding a rule whose priority is strictly less than the priority of every rule that already matches the request does not change the decision.*

The starter supplies two append lemmas:

- `bestRule_append_nonmatching`;
- `bestRule_append_dominated_decision`.

Add the two preconditions, write a one-sentence English justification for each, then case-split on whether the added rule matches:

```
lemma add_low_priority_stable(p: Policy, r: Rule, q: Request)
  requires matches(ruleMatcher(r), q) ==> /* TODO */
  requires matches(ruleMatcher(r), q) ==>
        /* TODO */
  ensures eval(Pol(p.rules + [r]), q) == eval(p, q)
```

The first clause prevents a new matching rule from becoming the only winner. The second prevents it from defeating the current winner. The assessed work is the justification and the short proof using the supplied append lemmas.

1.3 Normalization to Negation Normal Form [2.5 val.]

A matcher is in *negation normal form (NNF)* when `Not` only wraps an *atomic* matcher: `AnyReq`, `UserIs`, `ActionIs`, or `ResourceIs`. The starter supplies the predicate, the signatures, the lexicographic `decreases` clauses, and the structural `match` cases for mutually recursive `normalizeNot` and `normalize`.

1. Complete the marked branches: push `Not` inward using De Morgan's laws and eliminate double negation.
2. Prove **soundness**: `lemma normalize_sound(m: Matcher, r: Request)` with `ensures matches(normalize(m), r)== matches(m, r)`.
3. Prove **structural correctness**: `lemma normalize_is_nnf(m: Matcher)` with `ensures isNNF(normalize(m))`.

This is the project's advanced proof task. The soundness and NNF lemmas for `normalizeNot` and `normalize` support one another; follow the same cases as the functions.

1.4 Witness Lemma [2 val.]

State and prove the following theorem, **whose formal statement you must write yourself**:

> *If `eval(p, q) == AllowD`, then there exists an index `i` into `p.rules` such that:*
>
> - *`p.rules[i]` is an `Allow` rule whose matcher matches `q`, and*
> - *no `Deny` rule of priority **greater than or equal to** `rulePriority(p.rules[i])` matches `q`.*

Tie-breaking. Since `Deny` beats `Allow` at equal priority, the second condition must use $\geq$, not $>$. A condition using strict $>$ would miss the case where a same-priority `Deny` rule matches and would have defeated the `Allow` rule.

Translate this into a Dafny `lemma` with `exists`/`forall` quantifiers. Fill in the `ensures` clause in `allow_has_witness` and complete its supplied proof outline. It uses the supplied facts `eval_allow_selects_allow`, `bestRule_in_list`, and `bestRule_is_best`. Use `var i :| ...` to choose the index witness, then use `deny_not_beats_allow_is_lower` for the universal clause. Graders evaluate the formal statement and the proof.

1.5 Accumulator-Style Evaluator [2 val.]

Define a tail-recursive function:

```
function evalAcc(rules: seq<Rule>, q: Request, best: Option<Rule>):
   Decision
```

that scans the rules once, carrying the best matching rule found so far as an accumulator (`None` if no matching rule has been seen; `Some(rl)` otherwise, where `rl` is the highest-priority matching rule, with `Deny` winning ties).

Prove:

```
lemma evalAcc_eq_eval(p: Policy, q: Request)
  ensures evalAcc(p.rules, q, None) == eval(p, q)
```

Scaffold. The starter supplies the strengthened helper `evalAcc_eq_bestRule` and the algebraic lemma `combineBest_assoc`. Implement the scan, prove the supplied helper by induction, then derive the stated theorem with `best := None`. This is the "generalize the IH" move from Lecture 1, at scale.

Part 2 — Imperative and Object-Oriented Core [9 val.]

2.1 Batched Decisions [2 val.]

Implement:

```
method batchDecide(p: Policy, qs: array<Request>)
    returns (out: array<Decision>)
  ensures fresh(out) && out.Length == qs.Length
  ensures forall i :: 0 <= i < qs.Length ==> out[i] == eval(p, qs[i])
```

Your method must use a `while` loop (not a functional map). Design the loop invariant; as a hint, it should talk about the prefix `qs[..i]` already processed.

2.2 `PolicyStore` Class [3 val.]

Specify and implement a class `PolicyStore` with dynamic frames (`Repr` set, `Valid()` predicate). The *abstract state* is `ghost var rules: seq<Rule>`. Its purpose is to provide the mutable boundary around a policy: clients can add, remove, and query rules without depending on how those rules are stored. The `Valid()` invariant connects that hidden concrete state to `rules`, which is the policy observed by `lookup` and by `DecisionCache`.

The class must expose:

- `constructor()` — empty store.
- `method add(r: Rule)` — append a rule.
- `method remove(i: nat)` — remove the rule at index `i`; precondition `i < |rules|`.
- `method lookup(q: Request)` returns a `Decision` equivalent to `eval(Pol(rules), q)`.

Representation constraint. The assessed implementation must store rules in a resizable array; its field has type `array<Rule>`. **Fields of type `seq<Rule>` or `seq<object>` are not accepted.** This item exists to exercise the array-and-frames material of Lectures 4 and 6; a `seq` field bypasses both. Your `Valid()` predicate must explicitly relate the concrete data to the ghost `rules` sequence.

Bonus: linked-list alternative [+1 val. bonus]. As a separate file, you may implement a `LinkedPolicyStore` using a `Node`-based linked list instead. It must offer the same abstract `rules` sequence and the same contracts for `add`, `remove`, and `lookup`; its `Valid()` predicate should relate the node chain to that sequence and its `Repr` frame. This is additional work, not a substitute for the assessed array implementation, and it need not be integrated with `DecisionCache`.

Note: `remove` never allocates, so its frame postcondition should be `ensures Repr == old(Repr)`, not `fresh(Repr - old(Repr))`.

Hint for `remove`. Before overwriting the array, take a ghost snapshot of its used prefix. Then shift the suffix after index `i` one position to the left. A useful loop invariant separates the already rebuilt prefix (the original prefix before `i`, followed by the shifted entries) from the untouched suffix, which still agrees with the snapshot. Finally decrement `count` and update the ghost `rules` sequence.

2.3 `DecisionCache` Class [3 val.] + [+1 val. bonus]

Specify and implement a class `DecisionCache` that wraps a `PolicyStore` and caches the result of `lookup` for recently seen `Request`s, up to a fixed capacity of 1024 entries. The cache is a performance optimisation only: at every public method boundary, it must be observationally indistinguishable from calling `store.lookup(q)` directly.

Required: eager invalidation. Calling either underlying store mutator **empties the cache**: `PolicyStore.add` appends a rule and `PolicyStore.remove` removes one. The class must expose:

- `constructor(s: PolicyStore)`
- `method lookup(q: Request) returns (d: Decision)`
- `method addRule(r: Rule)` — delegates to store, then invalidates.
- `method removeRule(i: nat)` — delegates to store, then invalidates.

Required behaviour. An entry associates one `Request` key with the `Decision` computed for it. On `lookup(q)`, search the cached keys first:

- On a *hit*, return the stored decision; do not call `store.lookup(q)`.
- On a *miss*, call `store.lookup(q)` and return that result. If there is spare capacity, also store the new key–decision pair.
- If the cache is full, returning the newly computed result is sufficient; no eviction policy is required.
- `addRule` and `removeRule` first mutate the store and then clear every cached entry. A decision computed for the old policy must never remain available after a mutation.

Use concrete arrays for cache keys and values (with a size field) and a ghost map such as `entries: map<Request, Decision>` as the abstract view of the cache. At every well-formed point, `Valid()` should establish all of the following:

- the store and cache arrays are in `Repr`, and `store.Repr <= Repr`;
- the arrays have capacity 1024 and `size` is within their bounds;
- the concrete key–value pairs and `entries` describe the same cached associations; and
- every `q` in `entries` satisfies `entries[q] == eval(Pol(store.rules), q)`.

The frame must also keep the cache's arrays separate from the store's representation: a store resize must not accidentally replace or modify a cache array. The 3 val. for this item are graded on this eager design.

Hint: inserting a missed lookup. After an unsuccessful scan of all `size` occupied slots, use the concrete/ ghost correspondence in `Valid()` to establish `q !in entries`. If `size < CACHE_CAPACITY`, save the old ghost map, write `q` and its computed decision into slot `size`, increment `size`, and update the map to `oldEntries[q := d]`. To re-establish `Valid()`, show that every old map entry is still represented in the old prefix of the key array, while the new

entry is represented by the newly written final slot. If the cache is already full, do none of these writes: the miss is still correct because `d` came from `store.lookup(q)`.

Bounded-integer hit counter. The cache must maintain a `bv32` field `hits` that counts cache hits since construction, with a *saturation* guard: when `hits == BV32_MAX`, the next hit leaves `hits` unchanged rather than overflowing. Increment it only on a cache hit; a miss leaves it unchanged, whether or not the missed result can be inserted. Use `satInc` for the update and state the saturation property as a postcondition of `lookup`:

```
ensures hits == satInc(old(hits))   // on a hit
     || hits == old(hits)           // on a miss (cache not consulted)
```

(the exact formulation is up to you; the grader evaluates whether the bv32 non-overflow property is stated and proved).

Bonus [+1 val. bonus]: lazy invalidation. Instead of (or in addition to) eager invalidation, implement a variant `DecisionCacheLazy` that carries a `ghost var version: nat` on the store, copies the version into each cache entry on insertion, and re-validates on read by checking whether the stored version matches the current store version. Verify it and submit it in a separate file `vame-lazy.dfy`.

2.4 Stress Lemma [1 val.]

State and prove:

> *After building a `PolicyStore` by adding rules `rs[0]`, `rs[1]`, ... in order and then calling `lookup(q)`, the decision returned equals `eval(Pol(rs), q)`.*

Translate this into a Dafny lemma. The proof should be short — most of the work is in getting the specifications of `add` and `lookup` tight enough that Dafny can compose them directly. Think of this item as a *specification quality check*: it is easy only when §2.2's specs are strong.

Running-Code Component [1 val.]

Place your final Dafny file at `project/vame.dfy`. This component also requires a small Python program named `client.py`. It is a client of the verified engine, not a reimplementation of its policy semantics.

Provided wrapper API. We provide `vame_api.py`, which wraps Dafny's generated Python classes and conversions. Your client must import only `VameCache` from it:

```
from vame_api import VameCache

cache = VameCache(initial_rules)
decision = cache.lookup(request)  # "AllowD", "DenyD", or "NoMatch"
cache.add_rule(rule)
cache.remove_rule(index)
```

The wrapper converts JSON-shaped rules and requests to Dafny values, then delegates mutations to the compiled `DecisionCache`. Thus `add_rule` and `remove_rule` preserve eager invalidation. Do not import `module_`, `_dafny`, `PolicyStore`, or `DecisionCache` directly.

Operation-bundle JSON schema. Every bundle passed to `client.py` has this top-level shape:

```
{
  "initialRules": [ RULE, ... ],
  "operations": [ OPERATION, ... ]
}
```

A `REQUEST` and `RULE` have the following fields:

```
REQUEST = {"user": STRING, "action": STRING, "resource": STRING}
RULE    = {"type": "Allow" | "Deny", "priority": NAT,
           "matcher": MATCHER}
```

Here `NAT` is a non-negative JSON integer. A `MATCHER` is exactly one of:

```
{"op": "AnyReq"}
{"op": "UserIs",     "name": STRING}
{"op": "ActionIs",   "act":  STRING}
{"op": "ResourceIs", "res":  STRING}
{"op": "And", "left": MATCHER, "right": MATCHER}
{"op": "Or",  "left": MATCHER, "right": MATCHER}
{"op": "Not", "inner": MATCHER}
```

An `OPERATION` is one of:

```
{"op": "lookup", "request": REQUEST}
{"op": "add",    "rule": RULE}
{"op": "remove", "index": NAT}
```

The supplied test bundles also contain `expectedDecisions`; it is checker metadata, and a client should ignore it. The wrapper validates rules and requests and reports clear errors for malformed input.

What `client.py` must do. Start from `client-template.py` (rename the copy to `client.py`). It receives one operation-bundle JSON filename as its sole command-line argument. The bundle contains `initialRules` and an ordered list of operations:

```
{"op": "lookup", "request": {"user": "...", "action": "...", "
   resource": "..."}}
{"op": "add",    "rule":    { ... rule object ... }}
{"op": "remove", "index":   0}
```

Your `run(bundle)` function must construct one `VameCache` from `initialRules`, execute those operations in order, and append one result for every `lookup`. `add` and `remove` produce no result themselves. On standard output, print exactly one JSON object:

```
{"decisions": ["AllowD", "DenyD", "NoMatch"]}
```

Do not print debugging text on standard output; diagnostics belong on standard error.

Build and check. From the `project/` directory:

1. **Verify:** `dafny verify vame.dfy` must report 0 errors.
2. **Build:** Dafny 4.9 appends `-py` to this output name.

```
dafny build --target:py --output:vame vame.dfy
mv vame-py vame
```

3. **Check your client:**

```
python3 tests/check_client.py client.py tests/operations-small.json
    tests/operations-medium.json
```

4. **Submit** the checker stdout as `runlog.txt`.

The 1 val. is awarded if the client checker passes both supplied operation bundles. `Driver.py` and `tests/run_tests.py` remain available as ungraded diagnostics for the compiled Dafny implementation.

> **This component does not gate the verification marks for Part 2.** A group that verifies everything but cannot get the Python compilation working still receives full marks for §2.1–§2.4. Only the 1 val. here is at stake.

Submission

Submit a single `.zip` file via Fénix by **TBD — see the course portal**.

Required contents:

- `vame.dfy` — your full verified solution.
- `client.py` — your operation-bundle client.
- `runlog.txt` — stdout from the client checker.
- `reflection.md` — one page ($\leq$500 words): which specification in this project was hardest to state correctly, and why? Required for submission but not numerically graded.

Optional (bonus):

- `vame-lazy.dfy` — lazy-invalidation cache (§2.3).
- `vame-linked-list.dfy` — linked-list store bonus (§2.2).

After submission, you may be asked to present your work. During the discussion you may be required to perform small changes to the submitted code; both group members must be able to explain every annotation.

Grading Summary

Item	Marks
§1.1 Evaluator + totality	1.5
§1.2 Stability under low-priority additions	2.0
§1.3 Normalization soundness	2.5
§1.4 Witness lemma	2.0
§1.5 Accumulator-style evaluator	2.0
§2.1 `batchDecide`	2.0
§2.2 `PolicyStore`	3.0
§2.3 `DecisionCache` (eager)	3.0
§2.4 Stress lemma	1.0
Running-code component	1.0
Reflection	required
Total	**20.0**
`DecisionCache` lazy variant (bonus)	+1.0
Linked-list `PolicyStore` alternative (bonus)	+1.0

Specification strength is graded explicitly. Removing an `ensures` clause to make a proof go through *passes the verifier but weakens the contract for every caller.* The grader will read spec strength on items 1.2, 1.4, 2.2, and 2.3 independently of whether the proofs verify.

Academic Integrity

Labs are individual; this project is done in groups of two. You may discuss concepts with other groups, but the Dafny code you submit must be your own group's work. Using AI coding assistants as a *learning aid* (understanding an error message, a Dafny construct, or a library function) is permitted. Both group members must be able to explain every annotation during the post-submission discussion.

Standard course plagiarism policy applies; see the course Fénix page.